

AI UTILIZATION & ENGINEERING REPORT

SOLDIER : A DAY

AI 활용 기술 문서

AI 도구 · 프롬프트 설계 · 활용 내역

“AI가 만든 것을 믿지 않는다. 대신 틀렸을 때 반드시 죽는 장치를 함께 만든다.”

대상 프로젝트	SOLDIER : A DAY — 18일 생존형 병영 협동 RPG
개발 기간	2026-07-30 ~ 2026-08-07 (9일)
총 커밋	317 커밋 · 124 브랜치
코드 규모	62,310 줄 (TypeScript · C# · Python)
문서 버전	v1.0 · 2026-08-07

01 요약 — 무엇을 AI로 했는가

이 프로젝트는 **9일 만에 62,310줄**을 만들었다. 그것을 가능하게 한 것은 "AI에게 코드를 시켰다"가 아니라 **AI를 병렬 작업자로 쓰고, 그 산출물을 기계적으로 검증하는 파이프라인을 함께 설계한 것**이다.

317 커밋 / 9일	37 병렬 작업 워크트리	531 자동 테스트	488 절차적 생성 에셋
AI 도구	역할	산출물	
Claude Code Opus / Sonnet	설계 · 구현 · 리뷰 병렬 오케스트레이션	전체 코드베이스 · 문서 · 테스트 · 맵 생성기	
Meshy AI Text-to-3D	3D 소품 에셋 생성	상호작용 소품 45종 + 지급 장비 6종 프롬프트 시트 (docs/MESHY_PROMPTS.md , 575줄)	
절차적 생성 AI가 쓴 생성기	2D 픽셀 에셋 전량	타일 18 · 벽 5종×16 · 소품 105 · 캐릭터 시트 49 = PNG 488장	

핵심 판단

중간에 **3D에서 2D로 전환**했다. Meshy로 뽑은 3D 에셋은 개별 품질은 좋았지만 45종의 **톤을 서로 맞추는 비용**이 웹 빌드 예산 (25MB)과 함께 감당이 안 됐다. 대신 **AI에게 "에셋"이 아니라 "에셋 생성기"를 쓰게 했다** — 팔레트 하나를 고치면 488장이 일관되게 다시 나온다. 이 전환이 이 프로젝트에서 가장 중요한 AI 활용 결정이었다.

02 Claude Code 병렬 오케스트레이션

단일 대화로 코드를 받아쓰지 않았다. **CLAUDE.md** 에 **역할 분리 규칙**을 정의해 AI가 스스로 발주 → 병렬 구현 → 리뷰 → 병합을 수행하게 했다.

STEP 1	STEP 2	STEP 3	STEP 4
Opus — PM	Sonnet ×N — 워커	Opus — Tech Lead	검증 파이프라인
요구 분석 작업 분해 완료 기준 작성	브랜치별 병렬 구현 테스트 작성 자체 리뷰	코드 리뷰 충돌 해결 병합 · 최종 QA	테스트 · 빌드 배포 해시 대조

■ 파일 소유권으로 충돌을 설계에서 제거한다

병렬 작업의 진짜 비용은 구현이 아니라 **병합 충돌**이다. 그래서 발주 단계에서 워커마다 **소유 파일**을 명시적으로 갈랐다. 실제 배치 예시:

워커	범위	소유 파일
W1	소품 세이딩 · 구운 그림자 제거 · 노말맵 산출	tools/sprites/** · Art/2d/{props,tiles}/**
W2	벽 정면 배치 · Y소트 · 런타임 그림자 · 바닥 AO	Net/WorldDepth.cs · BaseScene.cs (벽·소품 구역)
W3	Lit 머티리얼 전환 · Light2D 배치 · 주야 연동	Net/WorldLighting.cs · Sprite2DImport.cs

같은 파일을 두 워커가 만져야 하는 경우(`BaseScene.cs`)는 메서드 영역까지 나눠 지시하고 병합은 오케스트레이터가 전담했다. 워커는 각자 `git worktree`에서 작업하므로 서로의 작업 트리를 볼 수 없다 — 이번 프로젝트에서 그렇게 만든 워크트리 37개다.

워커 간 계약 (contract) 명시

병렬 작업자는 서로 대화할 수 없으므로, 접점을 **발주서에 값으로 못박았다**. 계약이 없으면 W1이 만든 파일을 W2가 못 찾는 일이 반드시 생긴다.

```
벽 정면 타일 : Art/2d/tiles/wall_{kind}_face.png · 32×26px · kind = interior/utility/outdoor/wood
노말맵      : 원본과 같은 폴더 + _n 접미사 · 원본과 같은 크기 · 아틀라스 제외
그림자      : 런타임 담당(W2) — 생성기는 굵지 않는다(W1이 제거)
광원 방향   : 좌상단 45° 고정 — 아트·런타임 그림자 모두 같은 방향
```

■ 세션 한도를 설계 변수로 다룬다

대형 병렬 배치는 API 사용 한도에 걸린다. 한도는 **워커가 먼저 죽고 메인 루프가 잠시 남는 패턴**이라, 죽은 뒤에 대응하면 늦다. 그래서 **배치를 띄우는 시점에 이어가기 예약을 미리 걸고**, 진행 상태를 대화 맥락이 아니라 `HANDOFF.md` 파일에 기록하는 규칙을 세웠다 — **요약이 아니라 파일이 진실이다**. 세션이 끊겨도 다음 세션이 그 문서만 읽고 이어붙는다.

03 프롬프트 설계 — Meshy AI 3D 에셋

3D 소품 생성에서 **프롬프트 v0.1이 통째로 틀렸고**, 그 실패에서 프로젝트 전체에 적용할 프롬프트 원칙을 얻었다.

■ 실패: 모델의 기본값은 서구 기준이었다

초안은 **Metal barracks locker** 같은 일반 명사로 작성했다. 결과는 미군 풋락커나 서양식 라커였다 — **한국군 관물대가 아니었다**. 45종 중 대부분이 같은 문제였다.

항목	v0.1 (실패)	v0.2 (수정)
관물대	서양식 2문 캐비닛	철제 0.8T · 침대 옆 바닥 설치 · 폭 50cm 세로 장방형 · 상단 선반 + 내부 행거봉 + 하단 전투화 칸
침대	2층 침대(bunk bed)	단층 철제 1인 침대 — 침대형 생활관 8인 1실 표준
소총	assault rifle → M4/AR-15가 나옴	K2 — 측면 접이식 개머리판 · 조가비형 폴리머 총열덮개 · 상부 총몸 일체형 가늠자
방독면	(누락)	K-5 — 서구식 단일 정화통과 달리 양쪽 뺨에 정화통 2개

■ 원칙 1 — 고유명사를 믿지 말고 형태를 함께 준다

핵심

Meshy가 **K2**, **K-511**, **PRC-999K** 같은 한국 제식명을 학습했다고 **가정하지 않는다**. 모든 프롬프트를 **제식명 + 형태 묘사**로 작성해, 이름을 몰라도 형태 서술이 결과를 끌고 가도록 만들었다.

■ 원칙 2 — 네거티브를 긍정문으로 치환한다

Meshy는 버전에 따라 네거티브 프롬프트 칸이 없다. 그래서 억제하고 싶은 것을 **긍정 서술**로 바꿔 본문에 넣었다.

```
South Korean military equipment, realistic proportions,  
plain unmarked surfaces without text or markings,  
single isolated object, plain empty background, studio lighting, game asset, PBR
```

plain unmarked surfaces without text or markings 는 "글자를 넣지 마라"의 긍정문 치환이다. 네거티브 칸을 못 찾아도 이 문구만으로 글자 생성이 억제된다.

■ 원칙 3 — 색을 톤으로 지정하지 않고 재질군으로 쪼갬다

"전부 올리브색"으로 뺨으면 병영이 통째로 초록이 된다. 실제 한국군은 물건 종류마다 색이 다르다. 재질군 5종을 정의해 항목마다 하나를 배정했다.

코드	대상	프롬프트 문구
MAT-A	병영 철제 가구	powder coated sheet steel, light warm grey paint, slight edge wear
MAT-B	야전 장비	olive drab painted metal, matte finish, light scuffs and rust
MAT-C	피복 · 천	Korean digital granite camouflage fabric in beige grey, dark olive green...
MAT-D	취사 · 위생	brushed stainless steel, matte, light scratches
MAT-E	종이 · 플라스틱 · 목재	matte plastic and paper, muted utilitarian colors

■ 원칙 4 — 시드를 고정하고, 2배로 뽑아 줄인다

시드를 고정해야 재생성 시 스타일이 유지된다. 폴리곤은 목표치의 2배로 생성한 뒤 줄인다 — 생성 모델은 목표 폴리곤에 맞춰 뽑으면 디테일부터 잃는다.

04 절차적 생성 — AI에게 에셋 대신 생성기를 쓰게 한다

2D 전환 후 PNG 488장을 손으로도, 이미지 생성 AI로도 만들지 않았다. AI가 Python 생성기 6,918줄을 작성했고, 그 코드가 에셋을 만든다.

■ 왜 이 방식이 이겼는가

문제	이미지 생성 AI	생성기 코드
105종 톤 일치	매번 다름 — 수동 보정 필요	팔레트 1개 공유 → 구조적으로 일치
타일 경계 이어짐	보장 불가	오토타일 16방향 규칙으로 생성
야간/흑한 밴드 6종	×6배 재생성	팔레트 시프트 1회
재현성	없음	2회 실행 해시 동일
수정 비용	해당 이미지만	어휘 1곳 → 전량 반영

■ 조각 어휘(parts) — 재사용 가능한 시각 부품

소품 105종 중 26종이 초기에는 `_box()` 하나를 공유했고, 그 함수는 단색 사각형 채우기가 전부였다. 그래서 관물대도 서류함도 식탁도 "색만 다른 네모"였다. AI에게 지시한 것은 "예쁘게 그려라"가 아니라 "조각 어휘를 만들고 그것을 조합하라"였다.

```
parts.py  조각 어휘 (공유)
slab 몸통 · grain(재질결: wood/metal/fabric/mesh) · panelize(문·서랍)
seam · handle(bar/knob/latch) · vent · rim · feet · label · wear

props_furniture.py / props_utility.py / props_outdoor.py  가족별 그림
각 모듈이 BUILDERS(이름 → f(w,h) → Image)를 내보내면
tiles.py가 그리는 함수만 같아 끼운다 — 타일 크기는 절대 안 바꾼다
```

결과: 105종 전부(100%) 조각 합성. 차량 = 차체+바퀴+적재함, 차단기 = 줄무늬 팔+경광등, 보일러 = 배관+바늘 달린 압력계. 모듈을 셋으로 나눈 이유도 병렬 작업 때문이다 — 세 워커가 같은 파일을 동시에 고치면 충돌한다.

05 AI 산출물을 어떻게 신뢰하는가 — 검증 장치

이 프로젝트의 핵심 방법론이다. AI가 만든 것을 눈으로 확인하지 않는다. 대신 틀렸을 때 반드시 죽는 자동 검사를 함께 만들게 한다.

검사	무엇을 막는가
결정성 해시	생성기를 2회 실행해 483장 PNG 해시가 같은지 확인. 다른면 어딘가에 비결정성이 섞인 것
팔레트 검사	산출물에 팔레트 밖 색이 하나라도 있으면 실패. 밴드 그레이딩이 팔레트를 통째로 미는 구조라, 이탈 색 하나가 6밴드를 전부 깨뜨린다
연결성 플러드필	생활관 스폰에서 벽·소품을 피해 걸어 모든 구역에 닿는지 실제로 탐색
sim ↔ 맵 대조	규칙(<code>zones.json</code>)이 선언한 인접 관계를 맵이 실제로 실현하는지 확인
이동 시간 대조	맵 실측 거리와 규칙의 이동 시간 표가 어긋나면 실패
531개 자동 테스트	기획서 수치 불변식 · 결론론 · 프로토콜 스키마 전건 검증
배포 해시 대조	배포마다 로컬/원격 번들 sha256을 비교 — "코드는 고쳤는데 화면은 옛것" 사고를 막는다

맵 생성기 하나에만 **자체 검사가 8개** 들어 있다. 이 검사들은 장식이 아니다 — 실제로 배포를 막고 버그를 잡았다.

■ 검증 장치가 실제로 잡은 사례

CASE 1 훈련장 10곳이 도달 불가였다

증상	사용자가 "훈련장으로 넘어갈 수가 없는데?"라고 신고
원인	맵 생성기 주석에 "정문(Z18)으로 나가서 간다"고 적혀 있었으나 정문 구멍을 뚫는 코드가 없었다 . 외곽 철조망이 부대를 빈틈없이 둘러쌌
파급	커리큘럼이 D-03부터 사격장을 배정 → 3일차부터 갈 수 없는 곳으로 가라는 필수 일과 → 사실상 진행 불가
교훈	주석은 검증이 아니다. 플러드필 검사를 상시 검사로 승격했다 — 도달 불가 10곳 → 0곳

CASE 2 AI 워커의 보고가 틀렸다

보고	워커: "로컬 광원 104개가 렌더에 전혀 기여하지 않는다"(ON/OFF 캡처 픽셀 동일)
실제	게임이 아니라 캡처 도구의 문제 . URP <code>Light2D</code> 는 <code>LateUpdate</code> 에서만 메시를 갱신하는데, 배치모드는 플레이 모드가 아니라 <code>LateUpdate</code> 가 돌지 않는다 → Point 광원 메시가 아예 생성되지 않았다
증거	썸 YAML에서 Point 광원의 <code>m_LocalBounds</code> 가 전부 0, Global만 정상. 리플렉션으로 강제 갱신 후 재촬영하니 ON/OFF 평균차 83.1
교훈	AI의 결론을 액면 그대로 받지 않는다 . 캡처 도구에 <code>-bakeLights</code> 플래그를 넣어 함정을 영구 차단

CASE 3 테스트가 버그를 정답으로 박아두고 있었다

증상	화면엔 "구제권 3장"이 뜨는데 눌러도 조용히 거부됨 — 18일 내내 죽은 기능
원인	<code>state.leaderId</code> 가 sim 어디에서도 할당되지 않아 항상 <code>null</code> . 프로토콜에 인텐트는 있는데 sim이 처리하지 않았다
함정	기존 테스트가 이 버그를 정답으로 단정하고 있었다 ("2/4 투표 후 leaderId는 null") → 테스트째 재작성
교훈	AI가 쓴 테스트는 현재 동작 을 고정할 뿐 의도 를 보장하지 않는다. 테스트도 리뷰 대상이다

CASE 4 조용한 no-op — 에러도 안 나고 사라진 코드

증상	소품 이음매·통풍살·다리 무늬가 결과물에서 사라짐. 에러는 없음
원인	<code>palette.neighbor(key, step)</code> 이 계열 끝에서 자기 자신을 반환(클램프). 이미 가장 어두운 색을 몸통으로 쓰면 디테일이 "같은 색으로 칠하기" = 보이지 않는 no-op
교훈	한 워커는 자기 파일에서만 우회했었다. 어휘를 중앙에서 고치고 전량 재생성해야 나머지도 반영된다 — 병렬 작업에서 우회는 부채가 된다

06 사람이 한 일 — AI가 못 한 것

정직하게 구분한다. AI가 코드를 썼지만, 무엇이 틀렸는지 알아챈 것은 대부분 사람이었다.

항목	주체	비고
구제권이 발동되지 않는다	사람	직접 플레이해서 발견. 에러가 안 나 자동 검출 불가였다
훈련장에 갈 수 없다	사람	플레이 중 신고 → 이후 플러드필 검사로 자동화
훈련일 시간이 모자란다	사람	체감으로 신고 → 실측하니 왕복 71초 vs 시간대 60초
"그림자가 전방면에 떠 있다"	사람	화면을 보고 지적 → 픽셀 분석으로 원인 특정
3D → 2D 전환 결정	사람	목업 4안 비교 후 채택. 방향 결정은 위임하지 않았다
구현 · 리팩터링 · 테스트 작성	AI	62,310줄 전부
원인 분석 · 픽셀 실측	AI	신고를 받은 뒤의 진단은 대부분 AI가 수행
검증 장치 설계 · 구현	AI	지시는 사람("검사를 함께 만들어라"), 구현은 AI

가장 중요한 관찰

AI는 "에러가 나는 문제"에 강하고 "조용히 잘못된 문제"에 약하다. 구제권이 죽어 있던 것, 훈련장에 못 가던 것, 시간이 모자라던 것 — 전부 예외도 로그도 남기지 않는 문제였고 전부 사람이 플레이하다 발견했다. 그래서 이 프로젝트의 대응은 "AI를 더 잘 쓰자"가 아니라 "조용한 실패를 시끄러운 실패로 바꾸는 검사를 늘리자"였다.

07 맥락 관리 — 긴 작업을 어떻게 이어붙였는가

9일 · 317커밋은 단일 대화 맥락에 담기지 않는다. 세 단계 방어선을 두었다.

순위	수단	역할
1차	자동 압축	대화가 길어지면 요약본으로 이어간다 (도구 기본 동작)
2차	HANDOFF.md	파일이 진실이다. 완료 · 진행 중 · 남은 작업 · 브랜치 상태 · 아직 눈으로 확인되지 않은 것을 기록
3차	docs/WORKORDER.md	A~H 발주서 원본. 목표 · 완료 판정 · 검증 방법 · 의존 · 중단 기준

워커에게는 최소 맥락만 준다 — 소유 파일 · 계약 · 검증 방법. 전체 히스토리를 주면 토큰만 쓰고 판단이 흐려진다. 긴 로그와 빌드 출력은 `tail` / `grep` 으로 요약해서만 받는다.

HANDOFF.md의 설계

이 문서의 핵심은 완료 목록이 아니라 "무엇이 아직 눈으로 확인되지 않았는지"다. AI는 "구현했다"와 "동작한다"를 구분하지 못하는 경향이 있으므로, 사람이 확인해야 할 체크리스트를 분리해 매 배치마다 누적한다.

08 재현 가능한 배포 파이프라인

AI가 코드를 고쳐도 **화면이 옛것이면 의미가 없다**. 이 저장소에서 그 사고가 반복돼 배포 절차 자체를 고정 규칙으로 만들었다.

1. python3 tools/sprites/generate.py	에셋 재생성 (결정성 해시 확인)
2. Unity -batchmode ... CreateScene	씬 생성 - 성공 신호 "Exiting batchmode successfully"
3. Unity -batchmode ... BuildPlayer.Web	WebGL 빌드 - error CS 0 확인
4. node tools/webgl-sync.mjs	빌드 산출물 → apps/web/public/game
5. npx vercel --prod --yes	웹 배포
6. sha256 로컬 vs 원격 대조	web.data.br 해시 일치 확인 ← 필수
7. git push	게임 서버(Render) 재배포

실제로 겪은 함정

이 저장소는 **웹(Vercel)**과 **게임 서버(Render)**가 따로 배포된다. 아트·씬만 바뀔 때는 Vercel만으로 충분해서 오래 문제가 없었는데, packages/sim 이 바뀐 배치에서 **git push** 를 빼먹으면 **새 클라이언트 + 옛 서버** 조합이 되어 수정이 아무 효과가 없다. 또 하나: **web.loader.js** 해시는 아트·씬만 바뀌면 **변하지 않는다** — 내용 검증은 반드시 **web.data.br** 로 해야 한다.

09 결론 — 이 프로젝트가 얻은 방법론

#	원칙	근거가 된 사건
1	AI에게 에셋이 아니라 생성기 를 시킨다	3D 45종 톤 정합 실패 → 2D 생성기 6,918줄로 488장을 일관 생성
2	병렬 작업은 파일 소유권 으로 충돌을 설계에서 없앤다	워크트리 37개 · 브랜치 124개를 충돌 없이 병합
3	워커 간 접점은 계약으로 값까지 못박는다	파일명·크기·좌표를 발주서에 명시
4	산출물이 아니라 검사를 함께 만들게 한다	맵 생성기 자체 검사 8종 · 테스트 531건
5	AI의 결론을 액면 그대로 받지 않는다	"광원이 동작하지 않는다" — 실제로는 캡처 도구 문제
6	맥락은 대화가 아니라 파일에 남긴다	HANDOFF.md — 세션이 끊겨도 이어붙는다
7	조용한 실패를 시끄럽게 만든다	사람이 잡은 버그는 전부 예외도 로그도 없던 것들이었다

이 문서의 모든 수치는 저장소에서 직접 측정했다 — 커밋·브랜치는 **git**, 코드 줄 수는 **wc -l**, 테스트 건수는 **npm run test** 실행 결과, 에셋 장수는 파일 시스템 집계다. 사례는 **HANDOFF.md** 와 커밋 이력에 기록된 실제 사건이다.